

Tarea 1: mishell, una shell simple en C

Sistemas Operativos 2026, Sección 2, Grupo 8

Integrantes: Joaquín Gustavo Adaúy Castillo, Isaac Amadeus Nelson Castro Villalobos,
Diego Felipe Fuentes Conejeros, Ignacio Esteban Placencia Palma

Repositorio: <https://github.com/SatonakiV/Tarea-Shell-S0>

1. Introducción

mishell es una shell de texto para Linux escrita en C (POSIX). Muestra un prompt con el directorio actual, lee una línea, la parte en comandos y la ejecuta. Soporta redirecciones (<, >, >>), pipes de largo arbitrario, background con &, los built-ins cd, exit, jobs, pmon (un mini top que lee /proc) y, como bonus, Ctrl+Z con fg y bg, y no se cae con Ctrl+C ni Ctrl+. Se compila con make (sin advertencias con -Wall -Wextra -std=gnu11) y se ejecuta con ./mishell (ver README.md).

2. Arquitectura general

2.1 Módulos

Tabla 1. Módulos del proyecto y su responsabilidad.

Archivo	Qué hace
shell.c	Ciclo principal, prompt, getline(), built-ins y señales de la shell. main.c solo llama a run_shell().
parser.c	Tokenizador propio: palabras y operadores (, <, >, >>, &, también pegados como echo hola>out), comillas y \, validación de sintaxis.
executor.c	fork(), grupos de procesos, pipes, dup2(), redirecciones y execvp().
jobs.c	Tabla de jobs, handler de SIGCHLD, espera del foreground y entrega de la terminal (tcsetpgrp()).
pmon.c	Lectura de /proc, cálculo de %CPU y refresco con alarm().

2.2 Ciclo principal

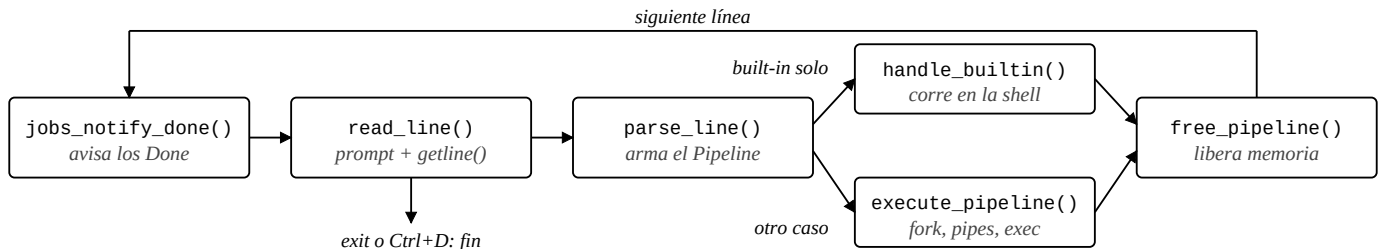


Figura 1. Ciclo principal de run_shell().

Un built-in corre en la shell solo si viene sin pipe ni &; lo demás va a execute_pipeline().

2.3 Estructuras de datos

- **Pipeline:** arreglo de Command, cantidad y flag background. Cada Command trae su argv terminado en NULL y sus Redirection (tipo + archivo). El parser reserva todo y run_shell() lo libera con free_pipeline().
- **Tabla de jobs:** arreglo estático jobs[64] en jobs.c. Cada Job guarda id, arreglo de PIDs del pipeline, pending (hijos que faltan), state (JOB_RUNNING, JOB_STOPPED o JOB_DONE, volatile sig_atomic_t), pgid, código y señal de término del último comando, modos de terminal, flag background y la línea (sin el &).

2.4 Decisiones de diseño

- Los pipelines en foreground *también* se registran como job. Así el handler de SIGCHLD es el que recoge a los hijos y el foreground solo espera a que su job termine o se detenga: no hay dos lugares peleando por los mismos hijos.
- Con terminal, cada pipeline tiene su propio grupo y el foreground recibe la terminal (ver 4.2).
- Los built-ins aceptan redirecciones (jobs > out.txt): se respalda el FD con dup(), se redirige, se ejecuta y se restaura.

3. Pipeline de N comandos

execute_pipeline() crea **los N-1 pipes antes de forkear**, hace fflush(NULL) (para que los hijos no hereden y repitan buffers de stdio) y bloquea SIGCHLD. Después crea un hijo por comando. El hijo i:

1. Entra al grupo del pipeline con `setpgid()` (lo crea el primero; el padre repite la llamada por si corre antes) y, si es foreground, toma la terminal con `tcsetpgrp()`. Luego desbloquea `SIGCHLD` y deja en `SIG_DFL` las señales que la shell ignora.
2. Hace `dup2(pipes[i-1][0], 0)` si no es el primero y `dup2(pipes[i][1], 1)` si no es el último.
3. Aplica sus redirecciones con `open() + dup2() + close()` (`O_RDONLY`, `O_WRONLY|O_CREAT|O_TRUNC` u `O_WRONLY|O_CREAT|O_APPEND`, modo 0644). Van después de los pipes, así un `>` explícito le gana al pipe.
4. Cierra los 2(N-1) extremos originales con `close_all_pipes()`: el 0 y el 1 ya apuntan a donde deben.
5. Llama a `execvp()`. Si falla imprime el error y hace `_exit(127)` (no existe) o `_exit(126)` (no ejecutable); un `open()` fallido hace `_exit(1)`. Es `_exit` y no `exit` para no vaciar buffers heredados del padre.

La shell cierra todos sus extremos apenas termina el ciclo de `fork()`. Con `cmd1 | cmd2 | cmd3` (pipes `p0` y `p1`) queda así:

Tabla 2. Descriptores de cada proceso de `cmd1 | cmd2 | cmd3` justo antes del `exec`.

Proceso	FD 0	FD 1	Extremos de pipe que conserva
cmd1	terminal	<code>p0[1]</code>	ninguno (ya duplicó el que usa)
cmd2	<code>p0[0]</code>	<code>p1[1]</code>	ninguno
cmd3	<code>p1[0]</code>	terminal	ninguno
shell	terminal	terminal	ninguno (los cierra tras los <code>fork</code>)

Si un `fork()` falla a mitad de camino, la shell mata con `SIGTERM` a los hijos ya creados (uno por uno: sin terminal el foreground comparte su grupo), los espera (sin zombies), recupera la terminal y termina, como pide R1.

4. Manejo de señales

4.1 SIGCHLD

Handler con `sigaction()`: `while ((pid = waitpid(-1, &st, WNOHANG | WUNTRACED | WCONTINUED)) > 0)` busca el job de cada PID. Si se detuvo marca `JOB_STOPPED`; si terminó, descuenta pending, guarda el código y la señal del último comando y marca `JOB_DONE` cuando pending llega a 0. Respalda `errno` y no imprime (`printf` no es `async-signal-safe`): el aviso (Done, Killed o Stopped) lo muestra `jobs_notify_done()` antes del próximo prompt. Va con `SA_RESTART` para no cortar el `getline()` del prompt. `SIGCHLD` queda bloqueado desde antes del primer `fork()` hasta que el job está registrado, así un hijo que muere muy rápido igual encuentra su job. El foreground espera con `while (state == JOB_RUNNING) sigsuspend(&old_mask)` (atómico, sin la carrera de `pause()`, y nunca se reinicia).

4.2 SIGINT y SIGQUIT

La shell las ignora con `sigaction()` y `SIG_IGN` (R6); como eso sobrevive al `exec`, cada hijo las devuelve a `SIG_DFL`. Con terminal, el foreground es el único grupo que la tiene (`tcsetpgrp()`): `Ctrl+C` le llega solo a él y puede leerla sin `SIGTTIN`. Al volver, la shell la recupera y restaura sus modos (`tcsetattr()`), para lo que ignora `SIGTTOU`. Los background nunca la tienen y `Ctrl+C` no les llega. Sin terminal (script por `stdin`) el foreground se queda en el grupo de la shell.

4.3 SIGTSTP y bonus fg/bg

La shell ignora `SIGTSTP`, así que `Ctrl+Z` solo detiene al foreground: el handler lo ve con `WUNTRACED`, el job queda en background detenido (con sus modos de terminal guardados) y se avisa `[n]+ Stopped. fg [n]` (o `%n`; sin número, el más reciente) le devuelve la terminal y sus modos, manda `SIGCONT` a su grupo y lo espera; `bg [n]` solo manda `SIGCONT`.

5. pmon

5.1 Campos de /proc

`pmon [segundos]` (2 por defecto) toma de `jobs.c` los PIDs de los background vivos o detenidos y lee:

- `/proc/[pid]/stat`: nombre entre paréntesis (se busca el último) con `strchr`, porque el nombre puede tener espacios), `state` (campo 3) y `utime` y `stime` (campos 14 y 15, en ticks).
- `/proc/[pid]/status`: la línea `VmRSS`: en kB.

5.2 %CPU y refresco

`%CPU` = $(\text{delta de utime+stime} / \text{sysconf}(_SC_CLK_TCK)) / (\text{delta de tiempo real}) \times 100$, con el tiempo real de `clock_gettime(CLOCK_MONOTONIC)` entre dos muestras seguidas. Para refrescar se usa `alarm(intervalo)` y un handler de `SIGALRM` que solo pone `actualizar = 1` (`volatile sig_atomic_t`); el ciclo espera con `sigsuspend()` y cada refresco limpia y redibuja la pantalla con `\033[2J\033[H` (si la salida es una terminal). Mientras corre, `pmon` instala un handler de `SIGINT` que marca `salir_pmon`; al salir hace `alarm(0)` y restaura el `SIG_IGN` de la shell y el `SIGALRM` previo, así `Ctrl+C` vuelve al prompt. Un proceso que termina deja de listarse (pregunta 5).

6. Pruebas realizadas

- **Ejecutor (make test):** 21 casos contra `execute_pipeline()`: códigos 127 y 126, muerte por señal (128+15), >, >>, <, combinadas, `open()` fallido, pipes de 2, 3 y 4, `yes | head -5`, comando intermedio fallido y código del último. Todos pasan.
- **pmon (tests/tests_pmon/):** lectura de `/proc` y %CPU contra un PID real (yes da cerca de 99%).
- **Shell completa:** sesiones en una pseudo-terminal (PTY) con la sesión de ejemplo de la pauta, errores de sintaxis, `cd`, `exit 300` (sale con 44) y `Ctrl+D`. Con `sleep 30 &`, `yes > /dev/null &`, `sleep 2 | sleep 3 &` y `python3` en background probamos `jobs`, `pmon 1`, `Ctrl+C` (prompt, `sleep 100` y `pmon`) y `Ctrl+\` (`sleep 100`): la shell y los background sobreviven. `cat`, `cat | wc -l`, `less` y `python3` en foreground leen la terminal, y `Ctrl+C` corta a `cat`. Bonus: `Ctrl+Z` sobre `sleep 100`, un pipeline, `vi`, `python3` y `less` detiene solo al foreground, `fg` y `bg` los reanudan y `cat &` avisa Stopped.
- **Zombies y descriptores:** un `sleep 1 &` que termina durante un `sleep 2` no deja hijos Z (`ps -ppid` desde afuera), y `/proc/<pid>/fd` de la shell muestra solo 0, 1 y 2 tras pipes y redirecciones fallidas.
- **fork() fallido, sanitizers y estrés:** con `ulimit -u` bajo, un pipeline de 12 `sleep` hace fallar un `fork()` y la shell avisa y termina sin dejar ningún `sleep` vivo. Con `make san (ASan y UBSan)` las sesiones no reportan nada, y `70 sleep 6 &` seguidos dan 64 jobs y 6 rechazados con aviso, todos sin zombies.

7. Limitaciones conocidas

Algunas cosas fallan dentro de lo que pide la pauta y otras simplemente quedaron fuera porque no se pedían.

- **Detalles dentro de lo pedido.** La tabla tiene 64 jobs (del background 65 en adelante la shell avisa y lo mata con `SIGTERM`); los built-ins solo andan sueltos (en `jobs | wc -l` la shell dice que jobs no existe); `pmon` muestra la letra de estado y el nombre corto, y la primera tabla sale tras el primer intervalo; como la shell ignora `SIGINT` y `SIGTSTP`, `Ctrl+C` o `Ctrl+Z` en el prompt no dibujan uno nuevo; con la tabla llena un foreground no se puede detener (se reanuda), y el binario `mishell` (viejo) está subido al repo.
- **Sin separadores.** No hay `;`, `&&`, `||` ni subshells (`echo uno; echo dos` imprime uno; `echo dos`).
- **Solo stdin y stdout.** No hay `2>`, `2>&1` ni `<<`; en `ls x 2> err`, el 2 le llega a `ls` como argumento.
- **Sin expansiones.** `$HOME`, `~` y los comodines llegan tal cual (`echo *.c` imprime `*.c` y `cd ~` falla), y no hay `$(...)`. Tampoco hay historial ni variables propias, que la pauta deja como bonus.
- **exit ignora los background.** Si quedan jobs vivos no avisa ni los termina: siguen corriendo huérfanos (a los detenidos el kernel les manda `SIGHUP`).

8. Preguntas de reflexión

1. **¿Por qué cd y exit son internos?** Cada proceso tiene su propio directorio de trabajo. Si `cd` corriera en un hijo, cambiaría el directorio del hijo, que muere enseguida, y la shell seguiría donde estaba. Con `exit` pasa lo mismo: terminaría el hijo.
2. **¿Qué pasa si no se cierran los pipes?** Si en `cmd1 | cmd2 | cmd3` la shell se queda con `p1[1]` abierto, `cmd3` (digamos `wc -l`) nunca recibe EOF porque para el kernel sigue habiendo un escritor. `wc` se queda bloqueado en `read()` y el pipeline se cuelga.
3. **¿Por qué waitpid en ciclo con WNOHANG?** Las señales no se encolan: si dos hijos mueren casi juntos (típico en un pipeline) puede llegar un solo `SIGCHLD`. Con una llamada se recoge uno y el otro queda zombie. El ciclo recoge todos los que hay y `WNOHANG` evita que el handler se bloquee cuando quedan hijos vivos.
4. **¿Y si no se restaura SIGINT en el hijo?** La shell ignora `SIGINT` y `SIGQUIT` (`SIG_IGN`), y esa disposición se hereda en el `fork()` y sobrevive al `exec` (solo los handlers vuelven a `SIG_DFL`). Sin `set_child_signals()`, el hijo heredaría `SIG_IGN` y no respondería normalmente a `Ctrl+C` ni a `Ctrl+\`: un `sleep 100` o `./programa_infinito` en foreground no se podría cortar y la shell se quedaría esperándolo. Por eso cada hijo las devuelve a `SIG_DFL` entre el `fork()` y el `execvp()`.
5. **¿Qué pasa si un proceso muere entre dos lecturas?** El handler de `SIGCHLD` lo recoge y `/proc/[pid]` desaparece, así que `fopen()` falla (o falta `VmRSS`, que un zombie no tiene). `tomar_muestra()` devuelve -1, la muestra se marca con `pid = -1`, el proceso no sale en la siguiente tabla y `pmon` sigue normal.

9. Conclusión

Lo central funciona: pipes sin FD filtrados, redirecciones, background sin zombies, `fork()` fallido bien cerrado, y `Ctrl+C` y `Ctrl+Z` que afectan solo al foreground gracias a los grupos y `tcsetpgrp()`. Tener un recolector principal de hijos (el handler de `SIGCHLD`) simplificó mucho la sincronización.